

Computer Networks: Lab 5 Socket programming-II

Reg no: 25MCB1010

Aim:

Write a program to implement TCP/UDP based single/multithreaded chat application using C/Python/Java.

Implement a client-server program using sockets to print the date/time of the remote server using Java & Python.

1. TCP Server and Client Program:

Implemented a basic TCP server that listens for incoming connections and a TCP client that connects to it. The server accepts a connection, exchanges data with the client, and demonstrates reliable communication using the TCP protocol. Executed both scripts with python3, showing how a client can send a message and the server responds.

tcp_server.py

```
bhaavya@bhaavya-VirtualBox:~/Desktop/CN/Lab5$ vim tcp_server.py
bhaavya@bhaavya-VirtualBox:~/Desktop/CN/Lab5$ cat tcp_server.py
import socket
server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_socket.bind(("localhost", 12345))
server_socket.listen(1)
print("Server started. Waiting for connection...")
conn, addr = server_socket.accept()
print("Connected by", addr)
while True:
    data = conn.recv(1024).decode()
    if not data:
        break
    print("Client:", data)

    msg = input("Server: ")
    conn.send(msg.encode())
conn.close()
```

tcp_client.py

```
bhaavya@bhaavya-VirtualBox:~/Desktop/CN/Lab5$ vim tcp_client.py
bhaavya@bhaavya-VirtualBox:~/Desktop/CN/Lab5$ cat tcp_client.py
import socket
client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client_socket.connect(("localhost", 12345))
while True:
    msg = input("Client: ")
    client_socket.send(msg.encode())
    data = client_socket.recv(1024).decode()
    print("Server:", data)
```

Execution:

Server side:

```
bhaavya@bhaavya-VirtualBox:~/Desktop/CN/Lab5$ python3 tcp_server.py
Server started. Waiting for connection...
Connected by ('127.0.0.1', 52000)
Client: Hello World
Server: How are you?
Client: I'm good, thank you!
Server: okay
Client: bye
```

Client Side:

```
bhaavya@bhaavya-VirtualBox:~/Desktop/CN/Lab5$ python3 tcp_client.py
Client: Hello World
Server: How are you?
Client: I'm good, thank you!
Server: okay
Client: bye
```

2. Multithreaded TCP Server and Client

Extended the TCP server into a multithreaded version (tcp_multithread_server.py). This allowed the server to handle multiple clients simultaneously, with each client connection managed in a separate thread. Tested it by running the server and multiple clients, confirming that concurrent communication worked correctly.

tcp_multithread_server.py

```
bhaavya@bhaavya-VirtualBox:~/Desktop/CN/Lab5$ vim tcp_multithread_server.py
bhaavya@bhaavya-VirtualBox:~/Desktop/CN/Lab5$ cat tcp_multithread_server.py
import socket
import threading
def handle_client(conn, addr):
    print("Connected:", addr)
    while True:
        try:
            msg = conn.recv(1024).decode()
            if not msg:
                break
            print(f"{addr}: {msg}")
            conn.send("Message received".encode())
        except:
            break
    conn.close()
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(("localhost", 12346))
server.listen(5)
print("Multi-threaded server started")
while True:
    conn, addr = server.accept()
    thread = threading.Thread(target=handle_client, args=(conn, addr))
    thread.start()
```

tcp_multithread_client.py

```

bhaavya@bhaavya-VirtualBox:~/Desktop/CN/Lab5$ vim tcp_multithread_client.py
bhaavya@bhaavya-VirtualBox:~/Desktop/CN/Lab5$ cat tcp_multithread_client.py
import socket
client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client_socket.connect(("localhost", 12346))
while True:
    msg = input("Client: ")
    client_socket.send(msg.encode())
    data = client_socket.recv(1024).decode()
    print("Server:", data)

```

Execution:

Server side:

```

bhaavya@bhaavya-VirtualBox:~/Desktop/CN/Lab5$ python3 tcp_multithread_server.py
Multi-threaded server started
Connected: ('127.0.0.1', 56760)
('127.0.0.1', 56760): Hello World
('127.0.0.1', 56760): How are you?
('127.0.0.1', 56760): I'm Bhaavya
('127.0.0.1', 56760): Bye

```

Client Side:

```

bhaavya@bhaavya-VirtualBox:~/Desktop/CN/Lab5$ python3 tcp_multithread_client.py
Client: Hello World
Server: Message received
Client: How are you?
Server: Message received
Client: I'm Bhaavya
Server: Message received
Client: Bye
Server: Message received

```

3. UDP Server and Client

Implemented a UDP server (udp_server.py) and a UDP client (udp_client.py). Unlike TCP, UDP is connectionless, so the client sends datagrams directly to the server without establishing a persistent connection. Running these scripts showed the difference between TCP's reliability and UDP's lightweight, faster communication.

udp_server.py

```

bhaavya@bhaavya-VirtualBox:~/Desktop/CN/Lab5$ vim udp_server.py
bhaavya@bhaavya-VirtualBox:~/Desktop/CN/Lab5$ cat udp_server.py
import socket
server_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
server_socket.bind(("localhost", 12347))
print("UDP Server started")
while True:
    data, addr = server_socket.recvfrom(1024)
    print("Client:", data.decode())
    server_socket.sendto("Message received".encode(), addr)

```

udp_client.py

```

bhaavya@bhaavya-VirtualBox:~/Desktop/CN/Lab5$ vim udp_client.py
bhaavya@bhaavya-VirtualBox:~/Desktop/CN/Lab5$ cat udp_client.py
import socket
client_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
while True:
    msg = input("Client: ")
    client_socket.sendto(msg.encode(), ("localhost", 12347))
    data, _ = client_socket.recvfrom(1024)
    print("Server:", data.decode())

```

Execution:

Server side:

```

bhaavya@bhaavya-VirtualBox:~/Desktop/CN/Lab5$ python3 udp_server.py
UDP Server started
Client: Hello
Client: Hello World
Client: I'm BHaavya
Client: okay bye

```

Client side:

```

bhaavya@bhaavya-VirtualBox:~/Desktop/CN/Lab5$ python3 udp_client.py
Client: Hello
Server: Message received
Client: Hello World
Server: Message received
Client: I'm BHaavya
Server: Message received
Client: okay bye
Server: Message received

```

4. Date-Time Server and Client

Created a Date-Time server and a client. The server sends the current system date and time to any connected client. I ran the programs to verify that the client successfully received and displayed the date-time information from the server.

dateTime_server.java

```

bhaavya@bhaavya-VirtualBox:~/Desktop/CN/Lab5$ vim datetime_server.py
bhaavya@bhaavya-VirtualBox:~/Desktop/CN/Lab5$ cat datetime_server.py
import socket
from datetime import datetime
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(("localhost", 9999))
server.listen(1)
print("Date-Time Server started")
conn, addr = server.accept()
current_time = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
conn.send(current_time.encode())
conn.close()

```

dateTime_client.java

```

bhaavya@bhaavya-VirtualBox:~/Desktop/CN/Lab5$ vim datetime_client.py
bhaavya@bhaavya-VirtualBox:~/Desktop/CN/Lab5$ cat datetime_client.py
import socket
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect(("localhost", 9999))
data = client.recv(1024)
print("Server Date & Time:", data.decode())
client.close()

```

Execution:

Server Side:

```

bhaavya@bhaavya-VirtualBox:~/Desktop/CN/Lab5$ python3 datetime_server.py
Date-Time Server started
bhaavya@bhaavya-VirtualBox:~/Desktop/CN/Lab5$

```

Client Side:

```

bhaavya@bhaavya-VirtualBox:~/Desktop/CN/Lab5$ python3 datetime_client.py
Server Date & Time: 2026-02-12 22:26:13
bhaavya@bhaavya-VirtualBox:~/Desktop/CN/Lab5$

```

5. Date Time Server (in java)

Implemented a simple server program in Java that listens on port 9999.

When a client connects, the server sends back the current system date and time using `new Date().toString()`. After sending the response, the server closes the connection.

This demonstrates how a server can provide real-time information to clients over a TCP socket. Compiled and run the server.

```

bhaavya@bhaavya-VirtualBox:~/Desktop/CN/Lab5$ vim dateTime_server.java
bhaavya@bhaavya-VirtualBox:~/Desktop/CN/Lab5$ cat dateTime_server.java
import java.net.*;
import java.io.*;
import java.util.Date;

public class dateTime_server {
    public static void main(String[] args) {
        try (ServerSocket serverSocket = new ServerSocket(9999)) {
            System.out.println("Server started. Waiting for client...");
            Socket socket = serverSocket.accept();
            System.out.println("Client connected.");

            PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
            out.println(new Date().toString());

            socket.close();
            System.out.println("Connection closed.");
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
bhaavya@bhaavya-VirtualBox:~/Desktop/CN/Lab5$ javac dateTime_server.java
bhaavya@bhaavya-VirtualBox:~/Desktop/CN/Lab5$ java dateTime_server
Server started. Waiting for client...
Client connected.
Connection closed.
bhaavya@bhaavya-VirtualBox:~/Desktop/CN/Lab5$

```

DateTime Client (DateTimeClient.java)

Implemented a client program that connects to the server on localhost:9999.

The client reads the message sent by the server (the current date and time) using a `BufferedReader` and prints it to the console. After receiving the response, the client closes the connection. Compiled and run the client while server is already running.

```

bhaavya@bhaavya-VirtualBox:~/Desktop/CN/Lab5$ vim dateTime_client.java
bhaavya@bhaavya-VirtualBox:~/Desktop/CN/Lab5$ cat dateTime_client.java
import java.net.*;
import java.io.*;

public class dateTime_client {
    public static void main(String[] args) {
        try (Socket socket = new Socket("localhost", 9999);
            BufferedReader in = new BufferedReader(
                new InputStreamReader(socket.getInputStream()))) {

            String serverResponse = in.readLine();
            System.out.println("Server Date & Time: " + serverResponse);

        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}

```

```

bhaavya@bhaavya-VirtualBox:~/Desktop/CN/Lab5$ javac dateTime_client.java
bhaavya@bhaavya-VirtualBox:~/Desktop/CN/Lab5$ java dateTime_client
Server Date & Time: Thu Feb 12 23:16:30 IST 2026
bhaavya@bhaavya-VirtualBox:~/Desktop/CN/Lab5$

```

Summary:

A TCP server and client in Python demonstrated reliable, connection-oriented communication, followed by a multithreaded TCP server that handled multiple clients concurrently. UDP server and client programs were then created to show connectionless, faster datagram-based communication, highlighting the contrast with TCP. A Date-Time server and client in Python and Java were developed, where the server sends the current system date and time to the client, illustrating a practical application of socket programming.

Submitted By: Patsamatla Bhaavya Ramita 25MCB1010

Email: Patsamatla Bhaavya Ramita 25MCB1010